



中山大学 软件工程学院

SUN YAT-SEN UNIVERSITY SCHOOL OF SOFTWARE ENGINEERING

SSE206 《计算机网络》

第三章 传输层

南雨宏

nanyh@mail.sysu.edu.cn

2026-04-15

第 3 章：提纲

3.1 概述和传输层服务

3.2 多路复用与解复用

3.3 无连接传输：UDP

3.4 可靠数据传输的原理

3.5 面向连接的传输：TCP

- 段结构
- 可靠数据传输
- 流量控制
- 连接管理

3.6 拥塞控制原理

3.7 TCP拥塞控制

3.8 传输层功能的演化

拥塞控制原理

- **拥塞**非正式的定义: “太多的数据需要网络传输, 超过了网络的处理能力”
- 与流量控制不同
- 拥塞的表现
 - 分组丢失(路由器缓冲区溢出)
 - 分组经历比较长的延迟(在路由器的队列中排队)
- 网络中前10位的问题!



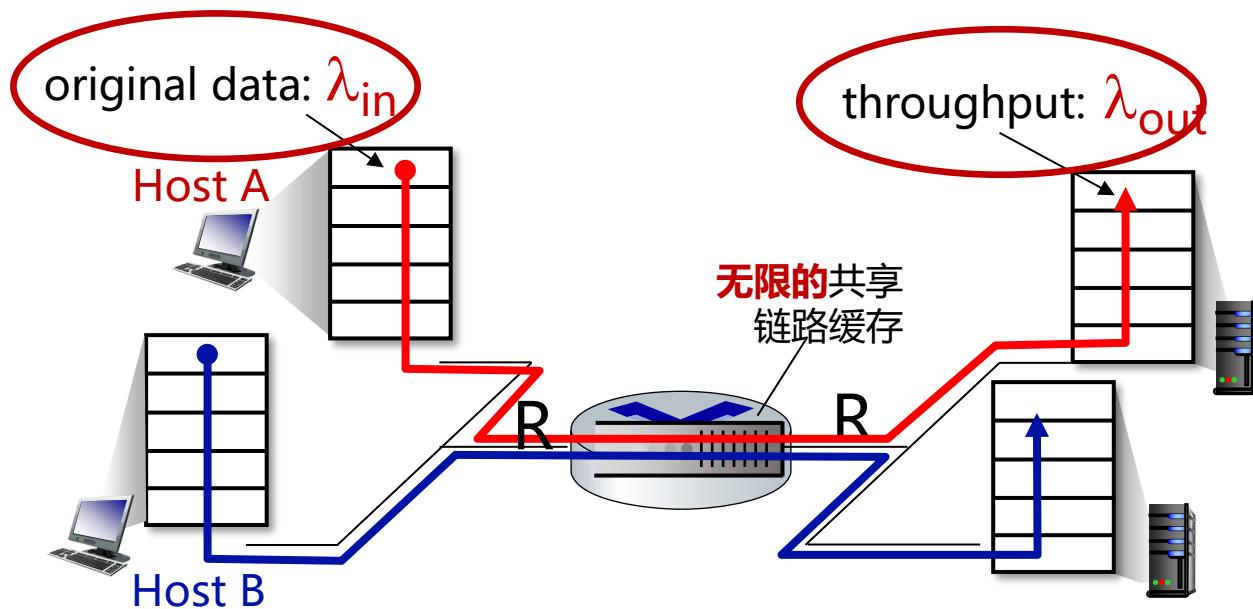
congestion control:
发送者太多, 发送速度太快



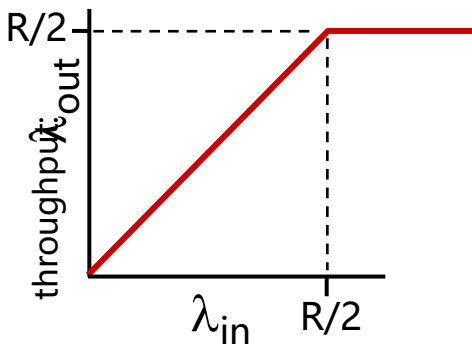
flow control: 一个发送者,
发送速度对接收方来说太快

拥塞的原因/代价: 场景1

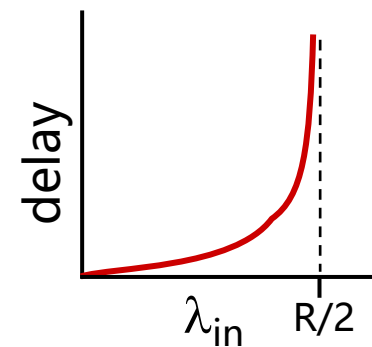
- ❑ 2个发送端, 2个接收端
- ❑ 一个路由器, 具备无限大的缓存
- ❑ 输出链路带宽: R
- ❑ 没有重传



Q: 当达到的速率 λ_{in} 接近 $R/2$ 时会发生什么?



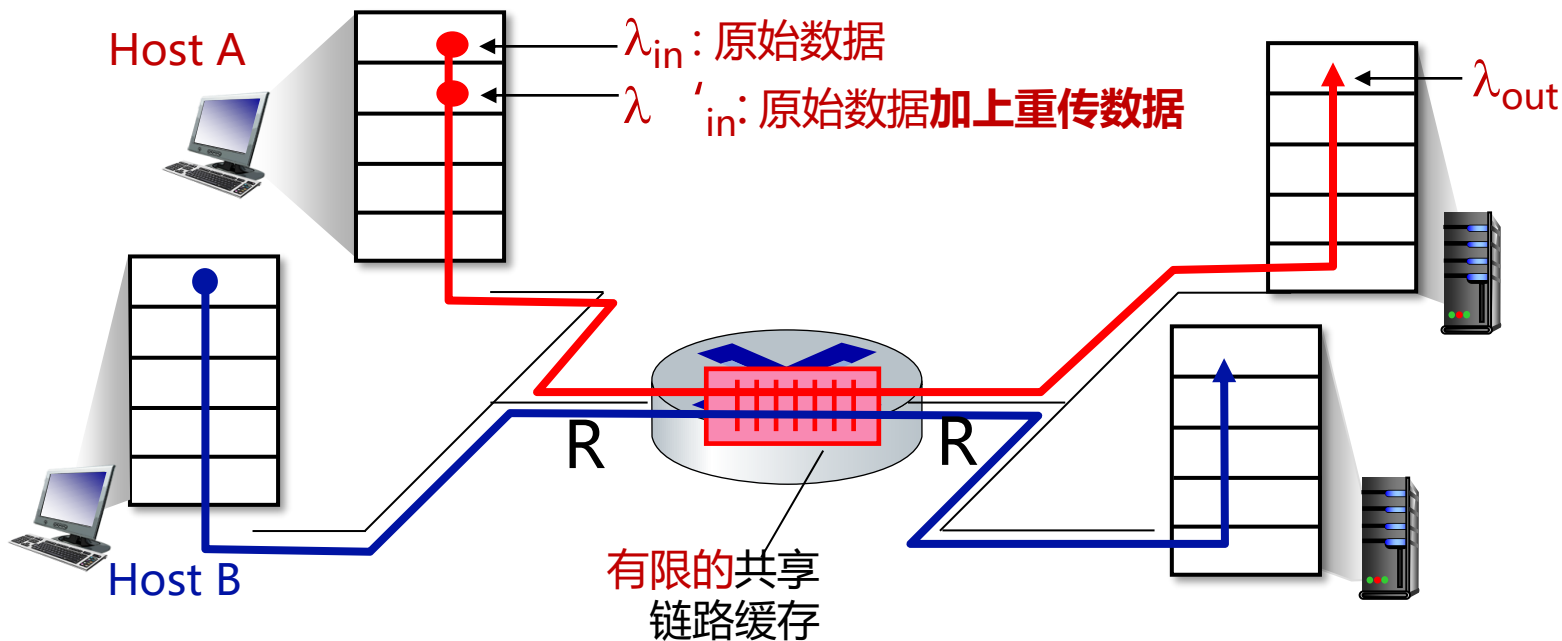
每个连接的最大吞吐量: $R/2$



当进入的速率 λ_{in} 接近链路带宽 R 时, 延迟增大

拥塞的原因/代价: 场景2

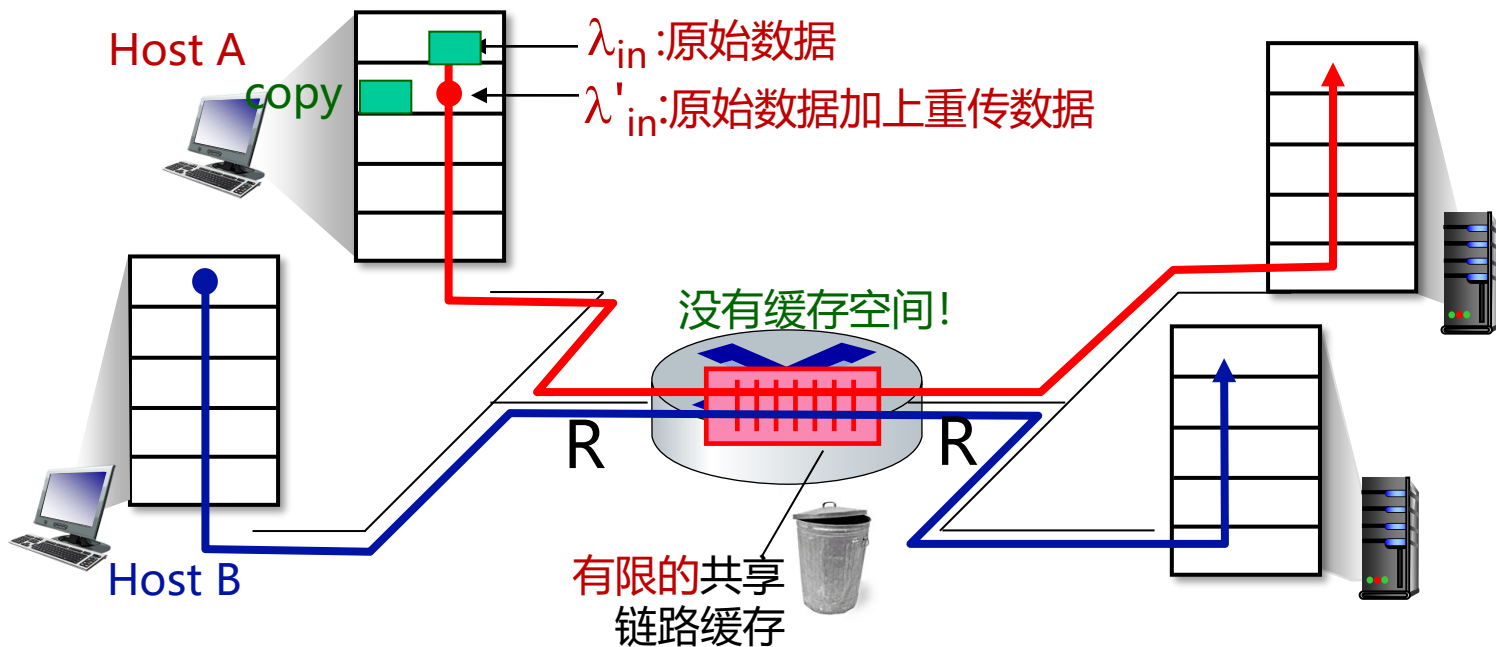
- 一个路由器，有限的缓存
- 分组丢失时，发送端重传
 - 应用层的输入=应用层输出: $\lambda_{in} = \lambda_{out}$
 - 传输层的输入，包括重传: $\lambda'_{in} \geq \lambda_{in}$



拥塞的原因/代价: 场景2

理想化: 掌握丢失信息

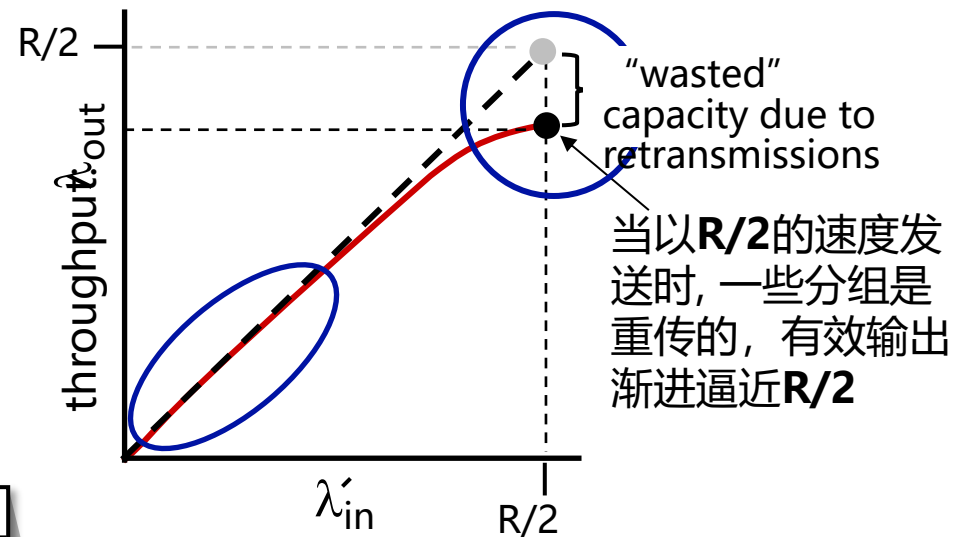
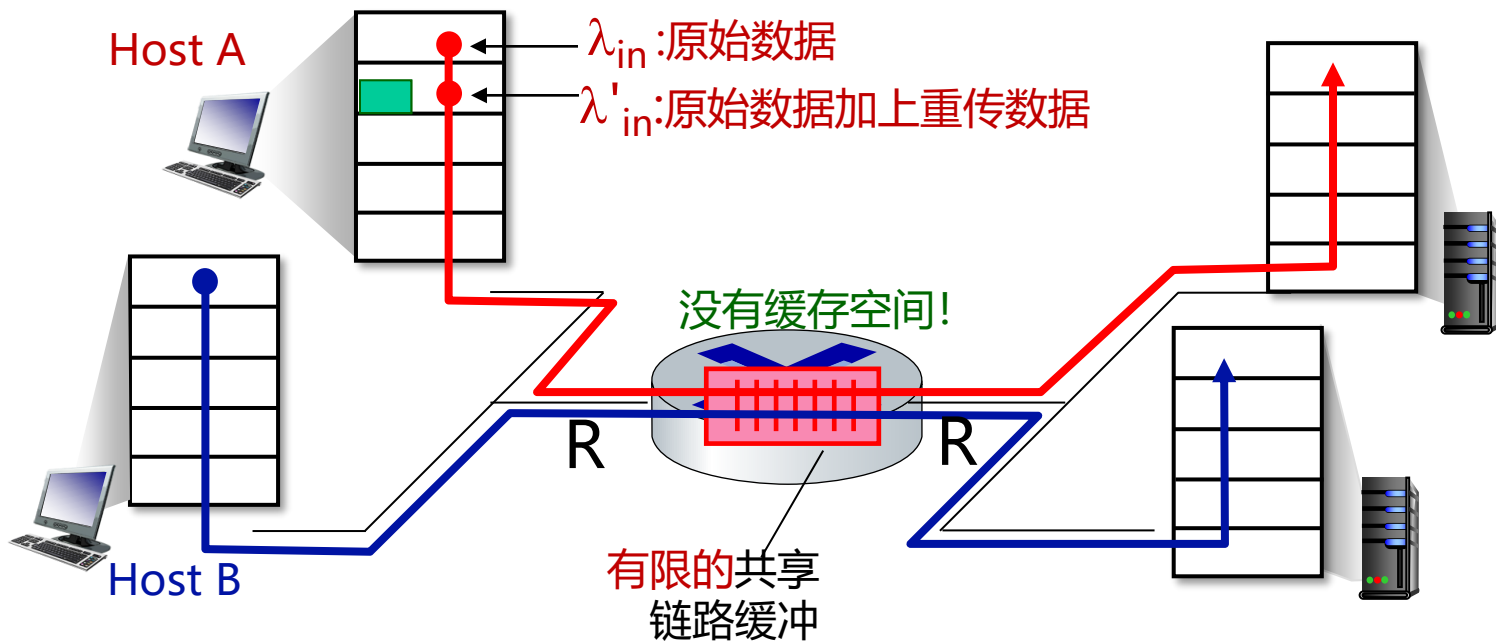
- 分组可以丢失，在路由器由于缓冲区满而被丢弃
- 如果知道分组丢失了，发送方重传分组



拥塞的原因/代价: 场景2

理想化: 掌握丢失信息

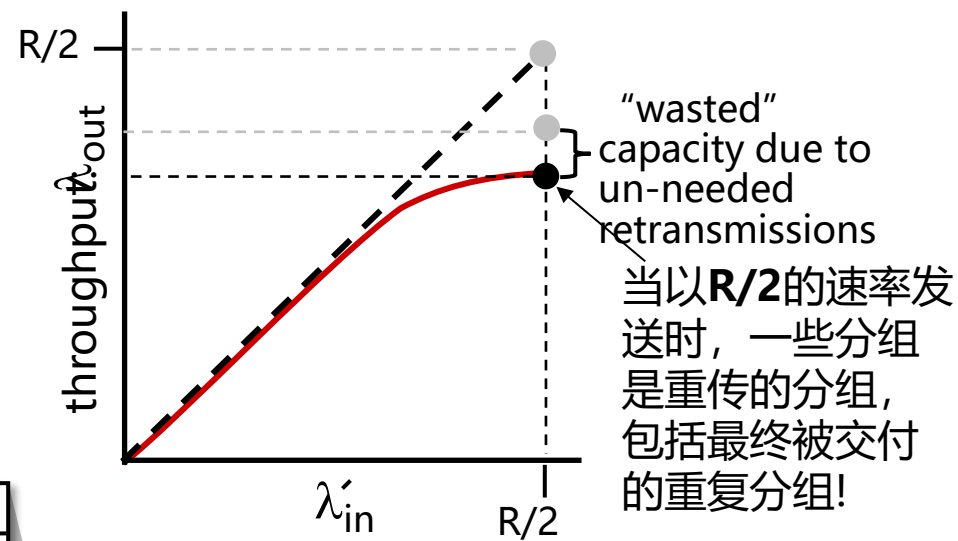
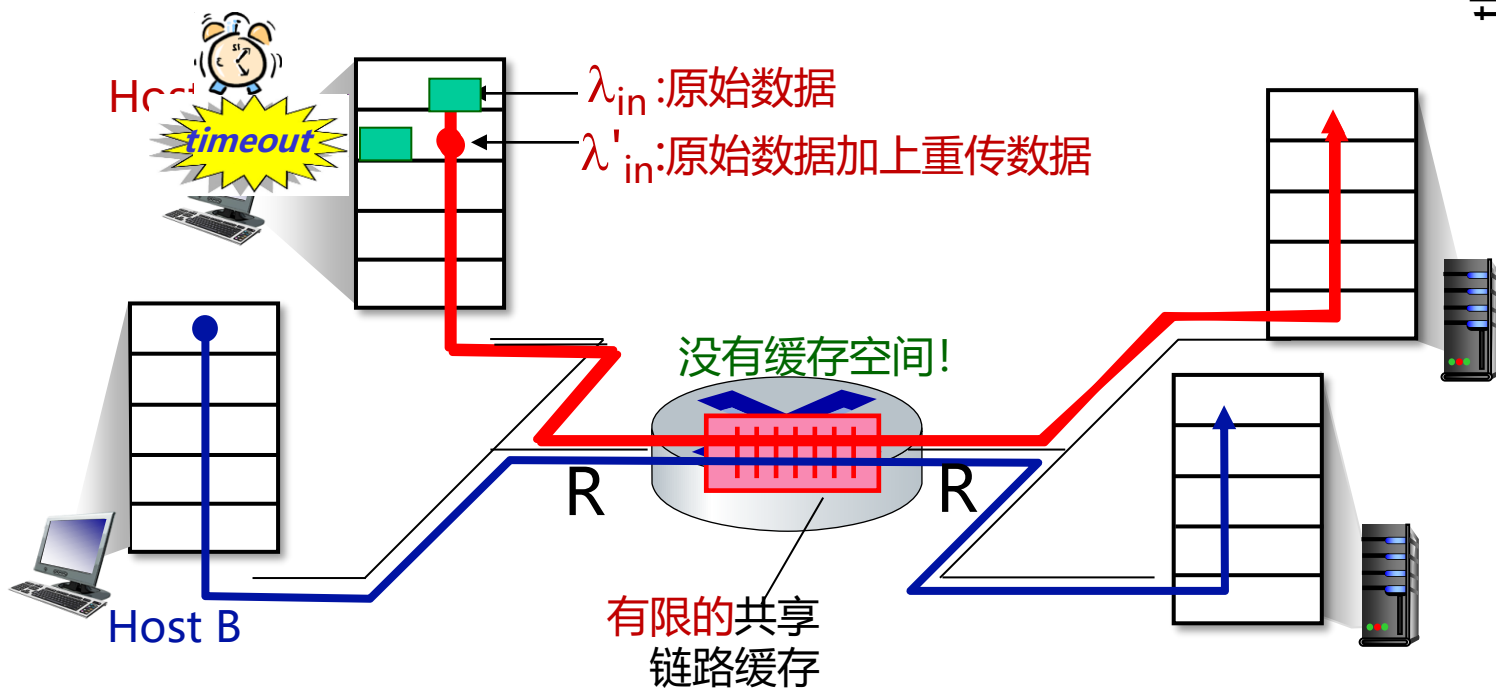
- 分组可以丢失, 在路由器由于缓冲区满而被丢弃
- 如果知道分组丢失了, 发送方重传分组



拥塞的原因/代价: 场景2

现实情况: 重复

- 发送端最终**超时**, 发送第**2**个拷贝
- 但实际上2个分组都成功被接收



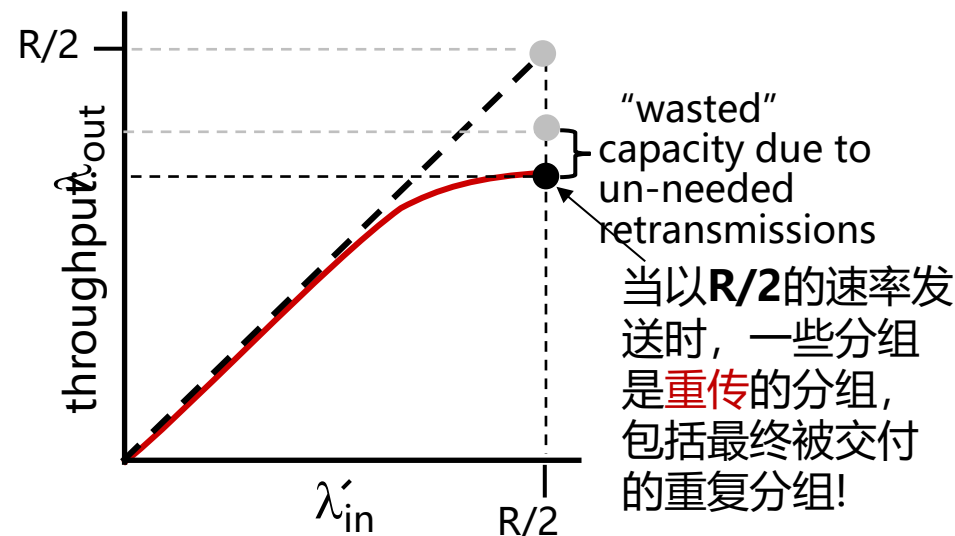
拥塞的原因/代价: 场景2

现实情况: 重复

- 发送端最终**超时**, 发送第**2**个拷贝
- 但实际上2个分组都成功被接收

拥塞的“代价”：

- 有必要的重传：为了达到一个有效输出，网络需要做更多的工作（重传）
- 没有必要的重传：链路中包括了多个分组的拷贝
 - 是那些没有丢失，经历的时间比较长（拥塞状态）但是超时的分组
 - 降低了的“goodput”



输出比输入少原因：

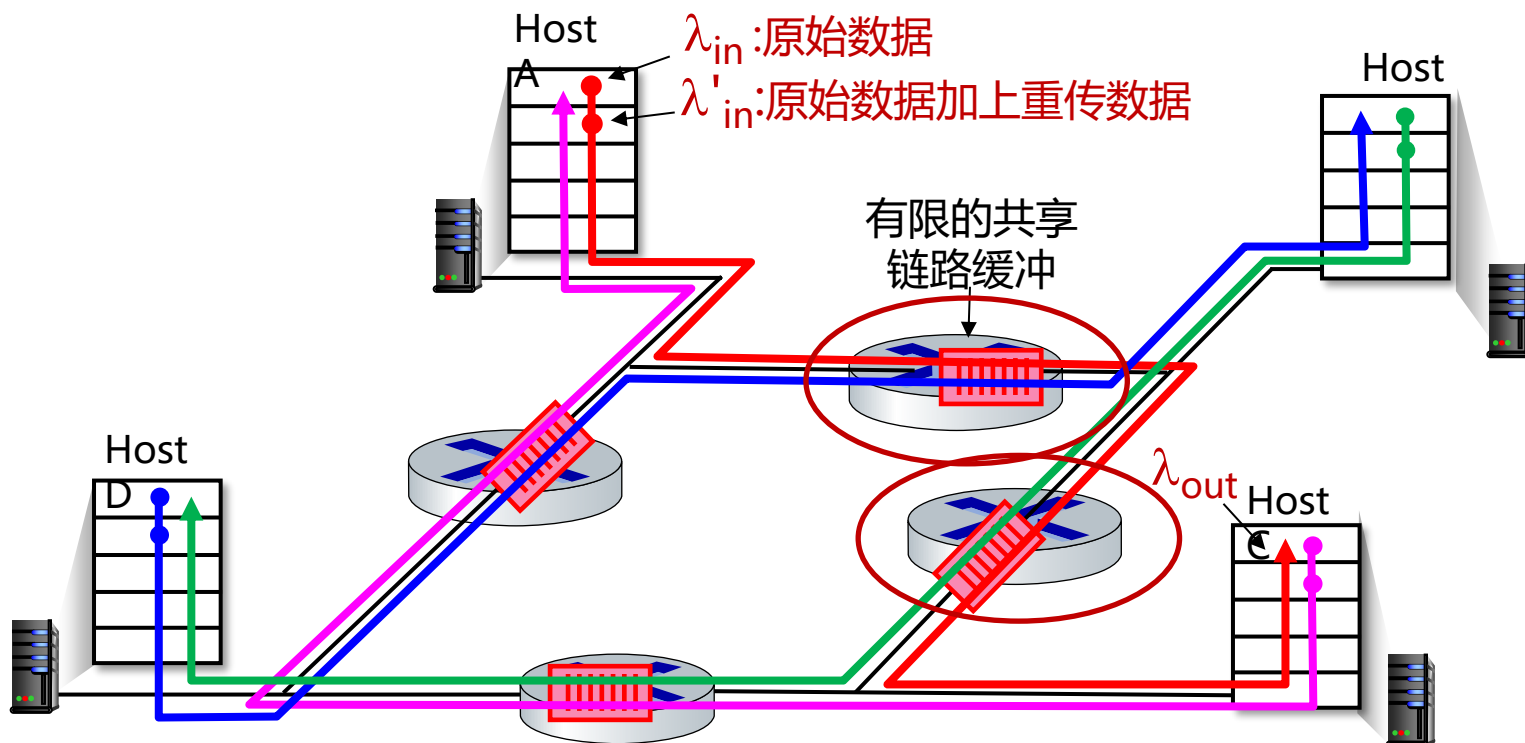
- 1) 重传的丢失分组；
- 2) 没有必要重传的重复分组

拥塞的原因/代价: 场景3

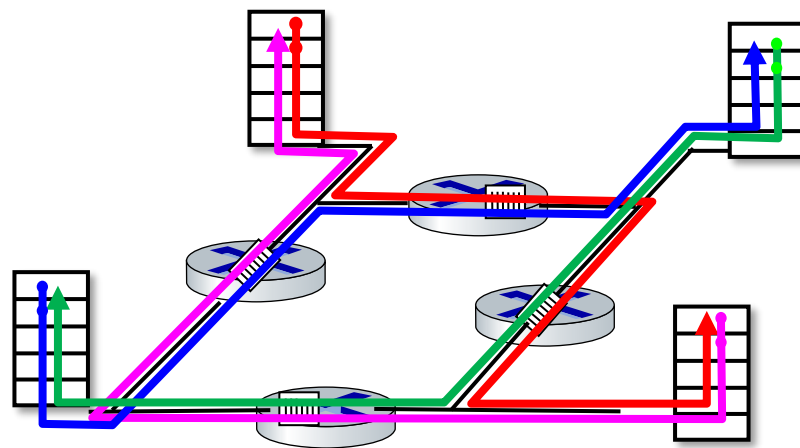
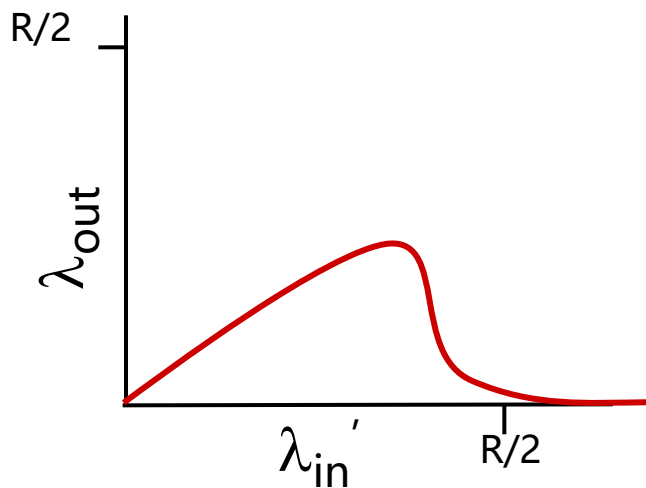
- 4个发送端
- 多重路径
- 超时/重传

Q: 当 λ_{in} 和 λ_{in}' 增加时, 会发生什么?

A: 当红色的 λ_{in}' 增加时, 所有到来的蓝色分组都在最上方的队列中丢弃了, 蓝色吞吐 $\rightarrow 0$



拥塞的原因/代价: 场景3

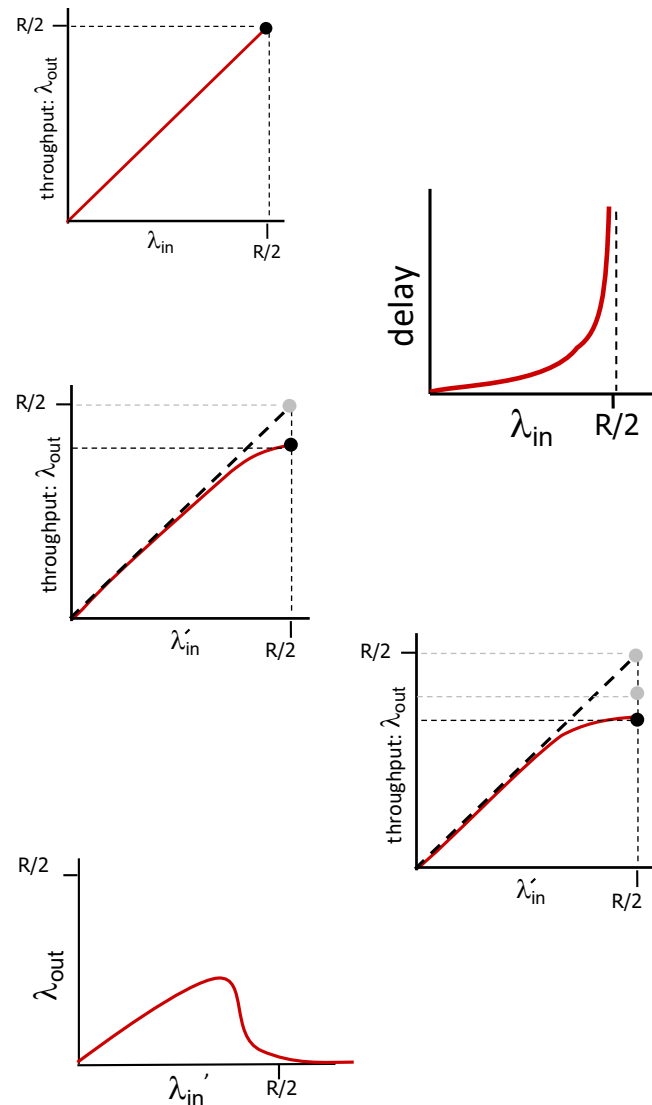


又一个拥塞的代价:

- 当分组丢失时, 任何 “关于这个分组的上游传输能力” 都被浪费了

拥塞的原因/代价: 见解

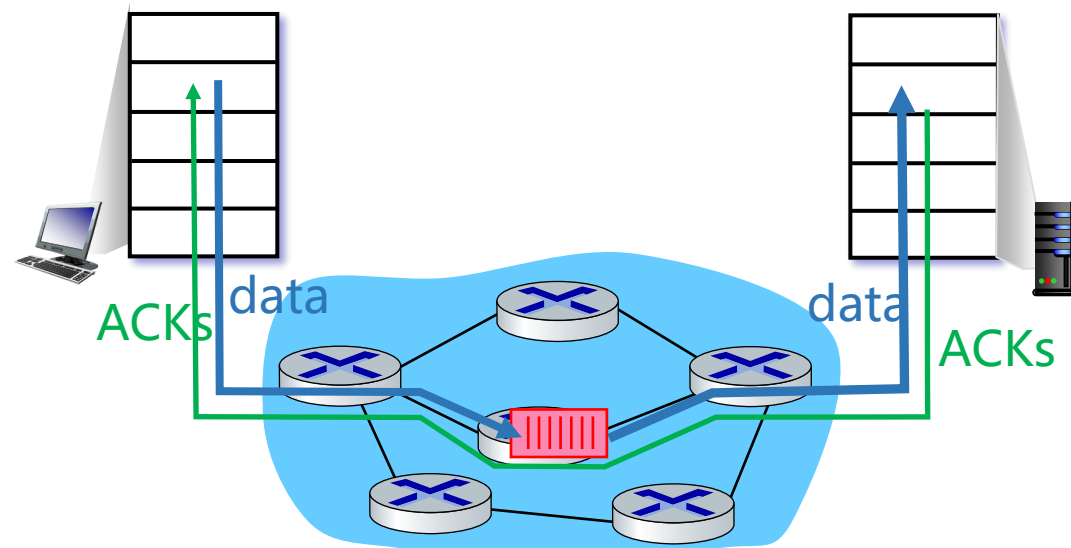
- 吞吐量永远无法超过容量
- 吞吐量在接近最大容量时会下降
- 丢包、重传降低了有效吞吐量
- 非必要重传进一步降低有效吞吐量
- 下游分组丢失时, 上游吞吐量被浪费



拥塞控制方法

端到端拥塞控制：

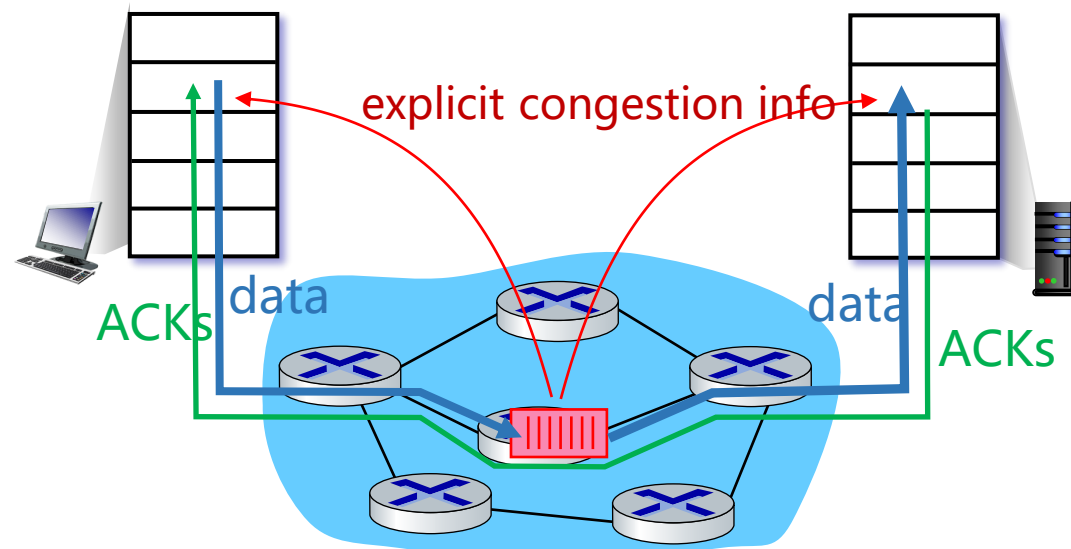
- 没有来自网络的显式反馈
 - 网络不提供任何内部信息表现拥塞状况，
需要靠自身来判断
- 端系统根据延迟和丢失事件推断是否有拥塞
- 经典TCP采用的方法* TCP Tahoe, Reno



拥塞控制方法

网络辅助的拥塞控制:

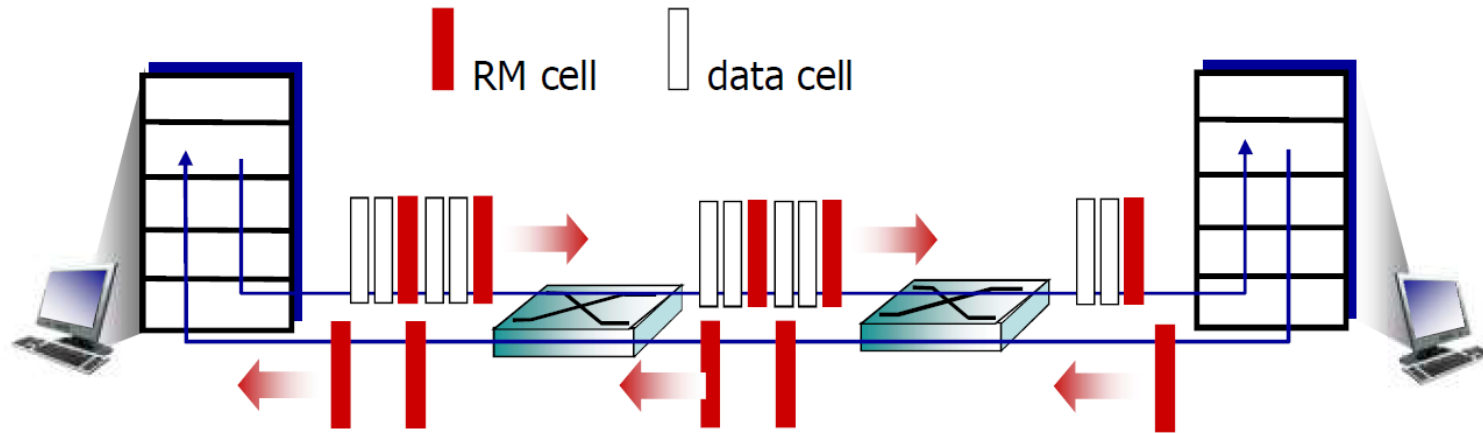
- 路由器提供给端系统以反馈信息
 - 单个 bit 置位, 显示有拥塞 (SNA, DECbit, TCP/IP ECN, ATM)
 - 显式提供发送端可以采用的速率



案例学习: ATM ABR 拥塞控制 (了解)

- ATM: Asynchronous Transfer Mode 异步传输模式
- ABR: available bit rate
 - “弹性服务”
 - 如果发送端的路径 “轻载”
 - 发送方使用可用带宽
 - 如果发送方的路径拥塞了
 - 发送方限制其发送的速度到一个最小保障速率上
- RM (资源管理) 信元
 - 由发送端发送,在数据信元中间隔插入
 - RM信元中的比特被交换机设置(“网络辅助”)
 - NI bit: no increase in rate (轻微拥塞)速率不要增加了
 - CI bit: congestion indication 拥塞指示
 - 发送端发送的 RM 信元被接收端返回,接收端不做任何改变

案例学习: ATM ABR 拥塞控制 (了解)



- 在 RM 信元中的2个字节 ER (explicit rate) 字段
 - 拥塞的交换机可能会降低信元中 ER 的值
 - 发送端发送速度因此是最低的可支持速率
- 数据信元中的 EFCI bit: 被拥塞的交换机设置成 1
 - 如果在管理信元 RM 前面的数据信元 EFCI 被设置成了 1, 接收端在返回的 RM 信元中设置 CI bit

第 3 章：提纲

3.1 概述和传输层服务

3.2 多路复用与解复用

3.3 无连接传输：UDP

3.4 可靠数据传输的原理

3.5 面向连接的传输：TCP

- 段结构
- 可靠数据传输
- 流量控制
- 连接管理

3.6 拥塞控制原理

3.7 TCP拥塞控制

3.8 传输层功能的演化

TCP 拥塞控制：机制

■ 端到端的拥塞控制

- 路由器不向主机发送反馈信息
 - 路由器的负担较轻
 - 符合网络核心简单的TCP/IP架构原则
- 端系统根据自身得到的信息，判断是否发生拥塞，从而采取动作

拥塞控制的几个问题：

■ 如何检测拥塞

- 轻微拥塞
- 拥塞

■ 控制策略

- 在拥塞发送时如何动作，降低速率
 - 轻微拥塞，如何降低
 - 拥塞时，如何降低
- 在拥塞缓解时如何动作，增加速率

TCP 拥塞控制：拥塞感知

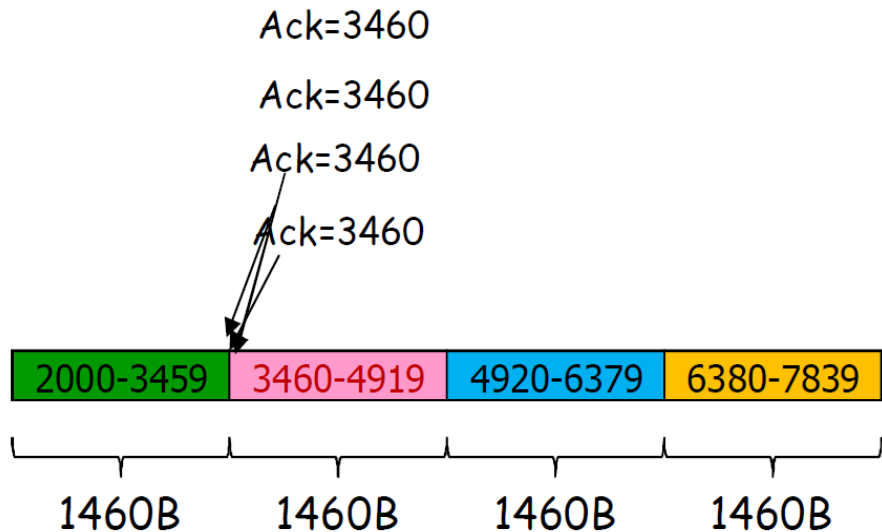
发送端如何探测到拥塞？

■ 某个段超时了（丢失事件）：拥塞

- 超时时间到，某个段的确认没有来
- 原因1：网络拥塞（某个路由器缓冲区没空间了，被丢弃）概率大
- 原因2：出错被丢弃了（各级错误，没有通过校验，被丢弃）概率小
- 一旦超时，就认为拥塞。可能有一定误判，但是总体控制方向是对的

■ 有关某个段的3次重复 ACK：轻微拥塞

- 第1个 ack，正常，确认绿段，期待红段
- 第2个重复 ack，意味着红段的后一段收到了，例如蓝段乱序到达
- 第2、3、4个 ack 重复，意味着红段的后第2、3、4个段收到了
- 橙段乱序到达，同时红段丢失的可能性很大（后面3个段都到了，红段都没到）
- 网络这时还能够进行一定程度的传输，**拥塞但情况要比第一种好**



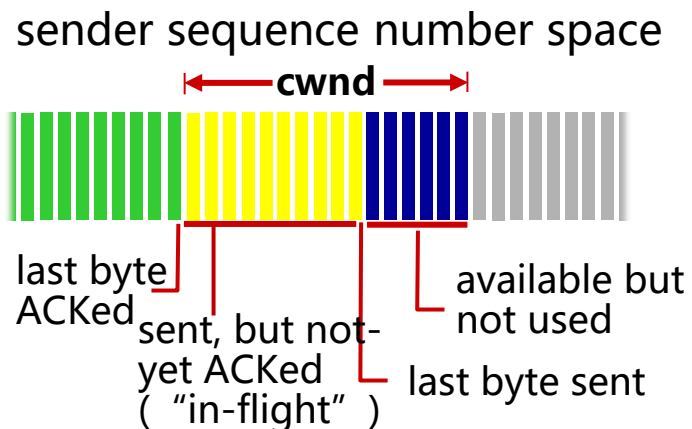
TCP 拥塞控制：速率控制方法

如何控制发送端发送的速率？

- 维持一个拥塞窗口的值：cwnd
- 发送端限制已发送但是未确认的数据量（的上限）：

$$\text{LastByteSent} - \text{LastByteAcked} \leq \text{Min}\{\text{cwnd}, \text{rwnd}\}$$

- cwnd：在未发生确认的情况下，发送方可以往出累计发送多少个字节
- cwnd 根据网络拥塞程度动态调整 从而粗略地控制发送方的往网络中注入的速率



TCP 发送行为：

- 粗略地说：发送 cwnd 字节，等待 RTT 的 ACKs，然后发送更多的字节

$$\text{TCP rate} \approx \frac{\text{cwnd}}{\text{RTT}} \text{ bytes / sec}$$

TCP 拥塞控制：速率控制方法

如何控制发送端发送的速率？

- cwnd 是动态的、感知网络拥塞程度的函数
 - 超时或者3个重复 ack, cwnd ↓
 - 超时时: cwnd降为1MSS, 进入慢启动 (**Slow Start, SS**) 阶段, 然后再倍增到 $cwnd/2$ (每个RTT), 从而进入拥塞避免 (**Congestion Avoidance, CA**) 阶段
 - 收到 3个重复 ack 时: cwnd 降为 $cwnd/2$, CA阶段
 - 否则 (正常收到ack, 没有发生以上情况): cwnd跃跃欲试↑
 - 慢启动阶段 **SS**: 加倍增加 (每个RTT $cwnd=cwnd*2$)
 - 拥塞避免阶段 **CA**: 线性增加 (每个RTT $cwnd=+1$)

TCP 拥塞控制：策略概述

拥塞控制策略:

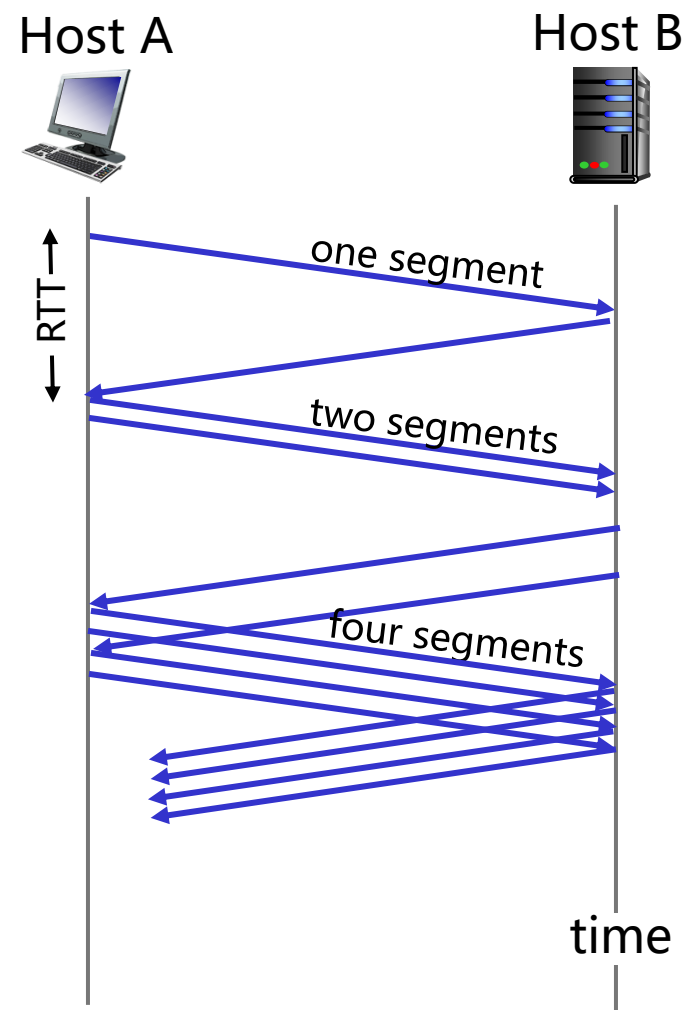
- 1. 慢启动
- 2. 拥塞避免 AIMD: *Additive Increase, Multiplicative Decrease*
 - 线性增、乘性减少 (三个冗余ack, threshold减半)
- 3. 快速恢复 (TCP Reno)
 - 超时事件后的保守策略 (TCP Tahoe)

TCP 慢启动 (冷启动)

- 连接刚建立, $cwnd = 1 \text{ MSS}$
 - 如: $MSS = 1460 \text{ bytes}$ & $RTT = 200 \text{ msec}$
 - 初始速率 = 58.4 kbps
- 当连接开始时, 指数性增加发送速率, 直到发生丢失的事件
 - 启动初值很低
 - 但是速度很快
- 可用带宽可能比 MSS/RTT 大的多
 - 应该尽快加速, 到达希望的速率

TCP 慢启动 (续)

- 当连接开始时，指数性增加（每个RTT）发送速率，直到发生丢失事件
 - 每一个 RTT，cwnd 加倍
 - 每收到一个 ACK 时，cwnd 加 1
 - 注：上述两句话等价（思考：Why）
- 慢启动阶段：只要不超时或收到3个重复ack，一个 RTT，cwnd 加倍
- 总结：**初始速率很慢，但是加速却是指数性的
 - 指数增加，SS 时间很短，长期来看可以忽略



TCP 拥塞控制: AIMD

- **方法:** 发送方可以提高发送速率直到发生丢包(拥塞), 然后在发生丢包(拥塞)事件时降低发送速率

加性增/线性增长

cwnd 的值以 1MSS 开始, 每一个 RTT 之后, 发送速率增加 1MSS, 直到检测出丢失事件

乘性减

在每次丢失事件时 (3个冗余 ACK), 将发送速率减半 ($cwnd/2$)

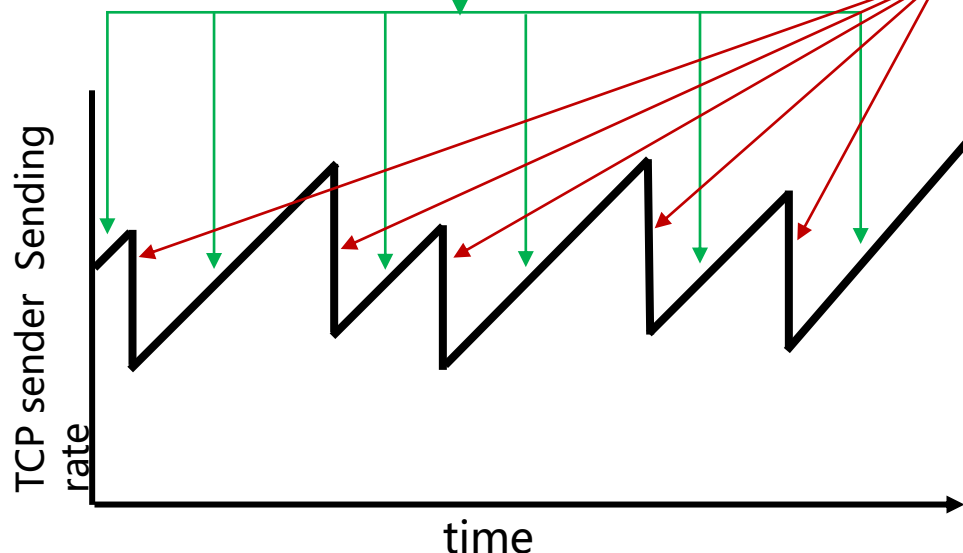
AIMD:

Additive Increase

(加性增)

Multiplicative Decrease

(乘性减)



AIMD 观察到
“锯齿” 行为: 带宽检测

TCP 拥塞控制: AIMD

思路

- 3个重复的 ACK 表示网络还有一定的段传输能力
- 超时之前的3个重复的 ACK 表示 “警报”

- 当收到3个重复的 ACKs:
 - cwnd 减半
 - 窗口（缓冲区大小）之后线性增长——拥塞避免
- 当超时事件发生时:
 - cwnd被设置成 1MSS, 进入慢启动SS阶段
 - 之后窗口指数增长
 - 增长到一个阈值（上次发生拥塞的窗口的一半）时, 再线性增加

TCP 拥塞控制: AIMD

乘性减 细节: 发送速率为

- 3次冗余 ACK, 就**减半** (TCP Reno)
- 超时 - 检测到丢失, 直接从 1MSS 开始慢启动
- 注: 早期版本TCP Tahoe, 3次冗余ACK或者超时, 都将拥塞窗口减少为1MSS

Why AIMD? 为什么这样设计?

- AIMD —— 一种分布式、异步算法, 已经被证明:
 - 特别适合整个网络, 整个网络全局的优化策略
 - 具有理想的稳定性

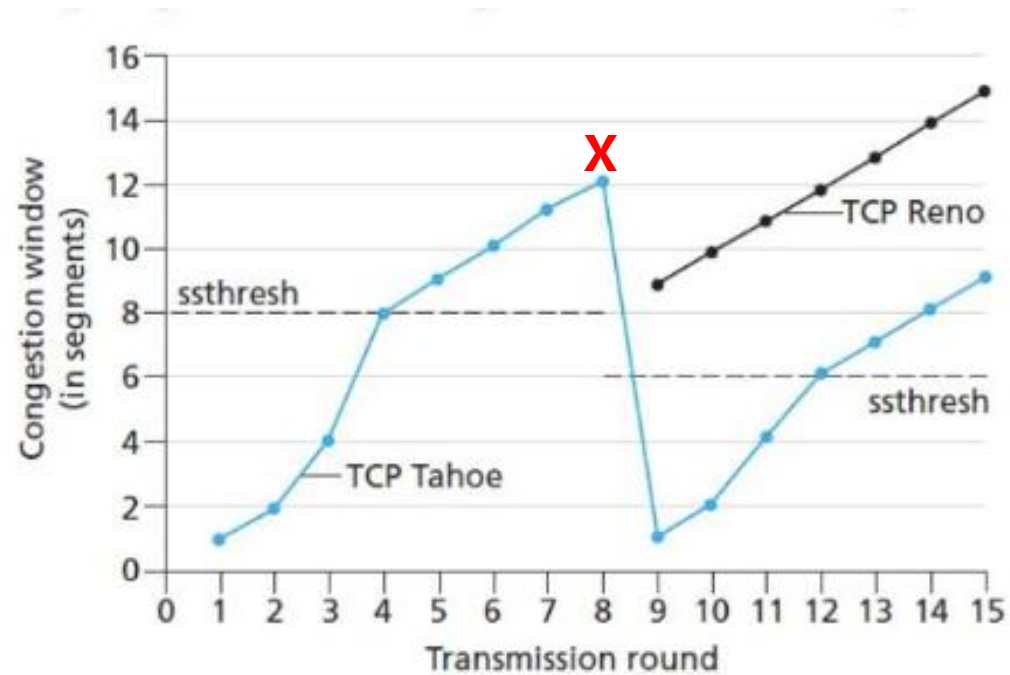
TCP: 从慢启动到拥塞避免

Q: 什么时候应该将指数性增长变成线性?

A: 在超时之前, 当 **cwnd** 变成上次发生超时的窗口的一半。

实现:

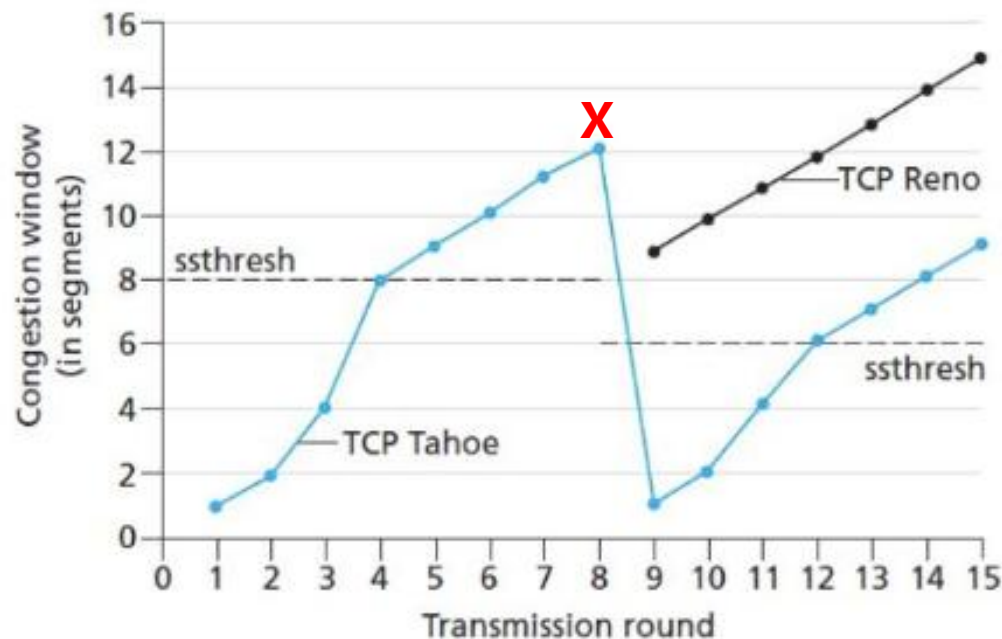
- 变量: **ssthresh** (“慢启动阈值” 的速记)
- 出现丢失, **ssthresh** 设置成 **cwnd** 的 1/2



* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/interactive/

TCP: 从慢启动到拥塞避免

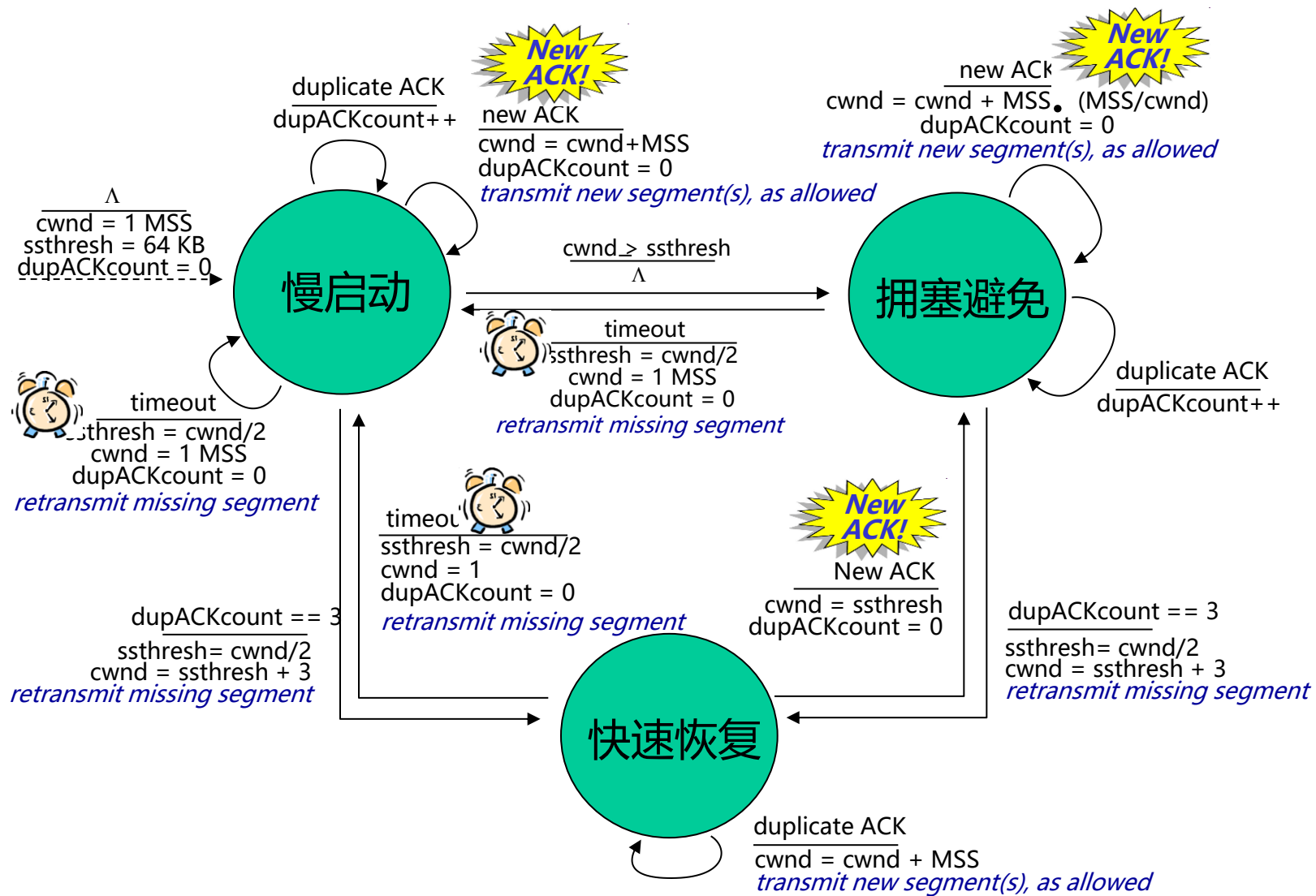
TCP Reno: 当连续接收到第三个重复的 ACK 时, 将 ssthresh 设置为当前拥塞窗口的一半 ($cwnd/2$, 但 $\geq 2MSS$) ; 重传丢失的报文段; 将 cwnd 设置为 ssthresh 加上 3MSS。这就使拥塞窗口通过报文段的数量增加了, 该数量是已经离开网络且其尾端被缓存的报文段数量 (3MSS) 。



总结: TCP 拥塞控制

- 当 $cwnd < ssthresh$, 发送端处于慢启动阶段 (**slow-start**), 窗口指数性增长
- 当 $cwnd > ssthresh$, 发送端处于拥塞避免CA阶段, 窗口线性增长
 - 当收到三个重复的 ACKs, $ssthresh$ 设置成 $cwnd/2$, 同时当前 $cwnd = ssthresh + 3MSS$ (三个ack)
- 当超时事件发生时 **timeout**, $ssthresh = cwnd/2$, $cwnd = 1MSS$, 进入 SS 阶段

总结: TCP 拥塞控制



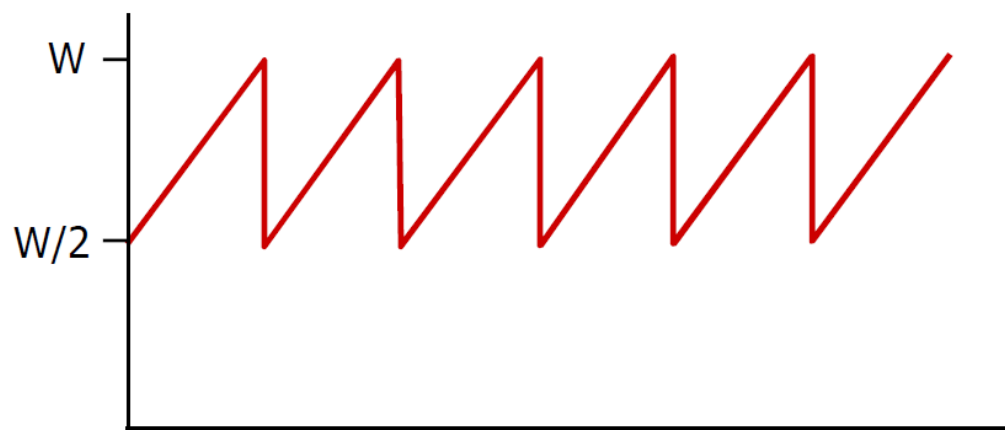
总结: TCP 拥塞控制 (发送端)

事件	状态	TCP发送端行为	解释
以前没有收到ACK的data被ACKed	慢启动 (SS)	$cwnd = cwnd + MSS$ $cwnd = cwnd * 2$ If ($cwnd > ssthresh$) 状态变成 "CA"	每收到一个ack, $cwnd + 1MSS$ 等同于每一个RTT $cwnd$ 加倍
以前没有收到ACK的data被ACKed	拥塞避免 (CA)	$cwnd = cwnd + 1$	加性增加, 每一个RTT对 $cwnd$ 加一个1 MSS
通过收到3个重复的ACK, 发现丢失的事件	SS or CA	$Threshold = cwnd / 2$, $cwnd = ssthresh + 3 * MSS$ 状态变成 "CA"	快速重传, 实现乘性的减. $cwnd$ 没有变成1 MSS
超时	SS or CA	$ssthresh = cwnd / 2$, $cwnd = 1 MSS$, 状态变成 "SS"	进入慢启动
重复的ACK	SS or CA	对被ACKed 的segment, 增加重复ACK的计数	$cwnd$ 和 $ssthresh$ 不变

TCP 吞吐量

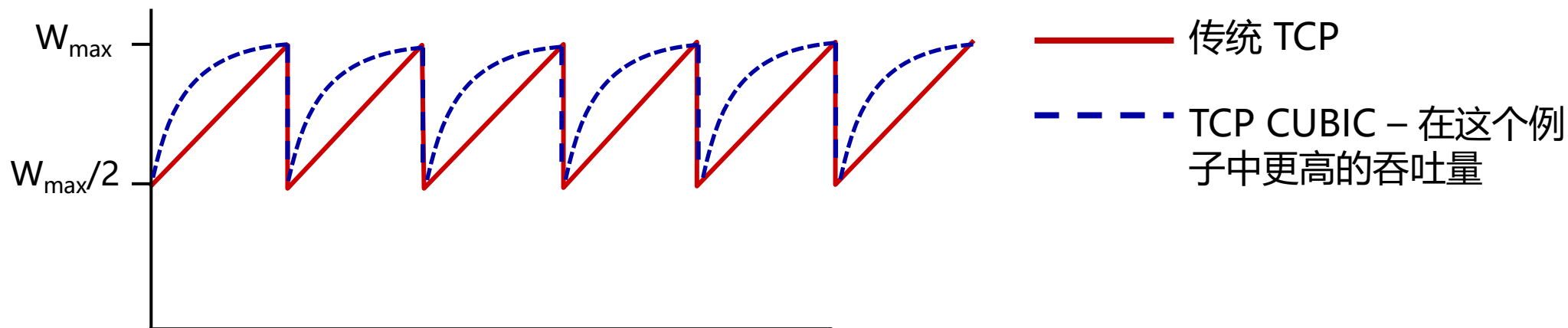
- TCP的平均吞吐量是多少，使用窗口尺寸 W 和 RTT 来描述
 - 忽略慢启动阶段，假设发送端总有数据传输
- W ：发生丢失事件时的窗口尺寸（单位：字节）
 - 平均窗口尺寸： $3/4W$
 - 平均吞吐量：

$$\text{avg TCP thruput} = \frac{3}{4} \frac{W}{RTT} \text{ bytes/sec}$$



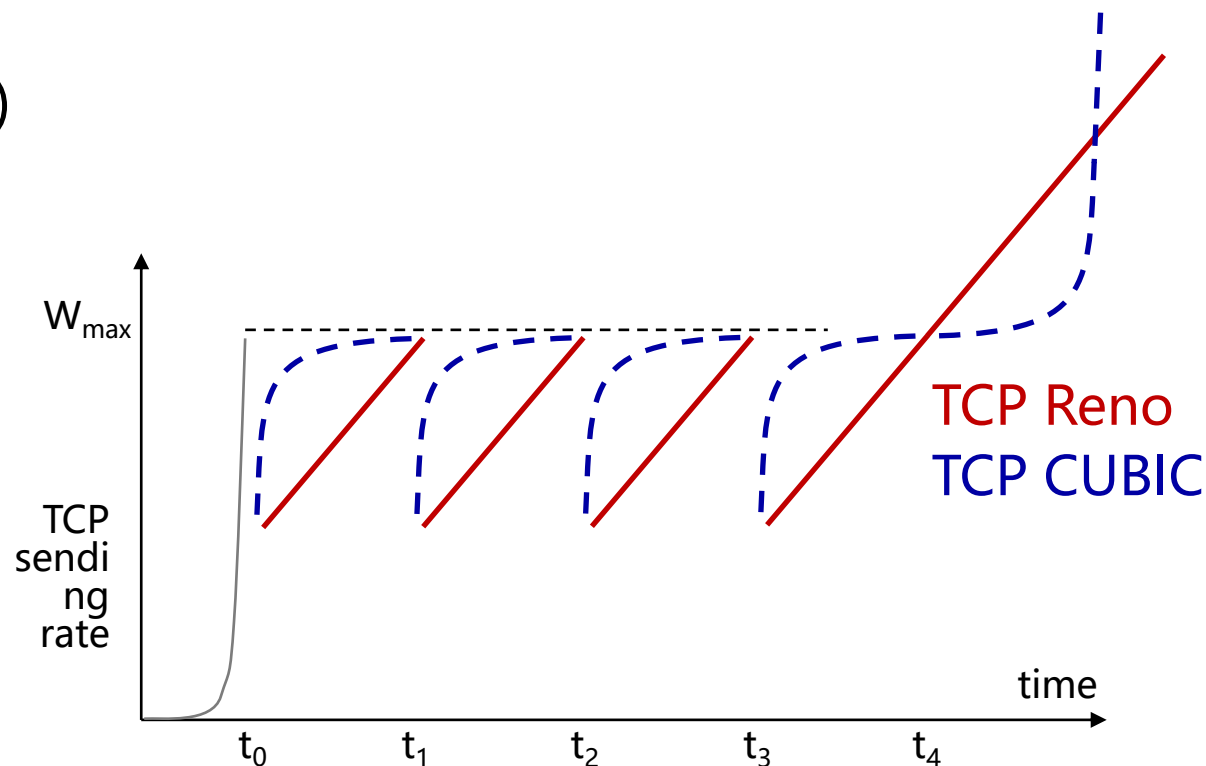
TCP CUBIC

- 有比 AIMD 更好的方法来“探测”可用带宽吗？
- 洞察/直觉:
 - $W_{\max}$: 检测到拥塞丢失时的发送速率
 - 瓶颈链路的拥塞状态大概率没有变化太多
 - 在丢失时将速率/窗口减半之后，最初更快地接近 $W_{\max}$ ，但随后更慢地临近 $W_{\max}$



TCP CUBIC

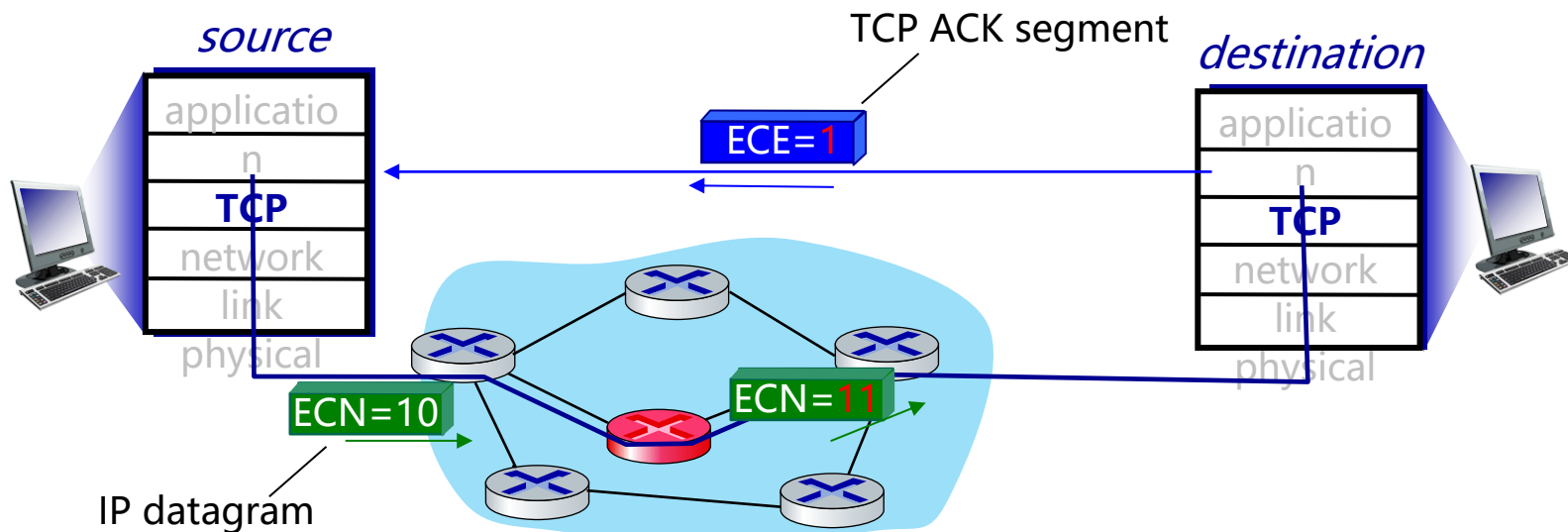
- K: 假定无丢包情况下, 当 TCP CUBIC 窗口长度将再次到达 $W_{\max}$ 时的未来时间点
 - K 本身是可调的
- 增加 W 作为当前时间 t 和 K 之间距离的立方的函数
 - 当离 K 越远时, 增加越大
 - 当接近 K 时, 增加较小(谨慎)
- 现代 Linux 默认版本



明确拥塞通告 (Explicit congestion notification, ECN)

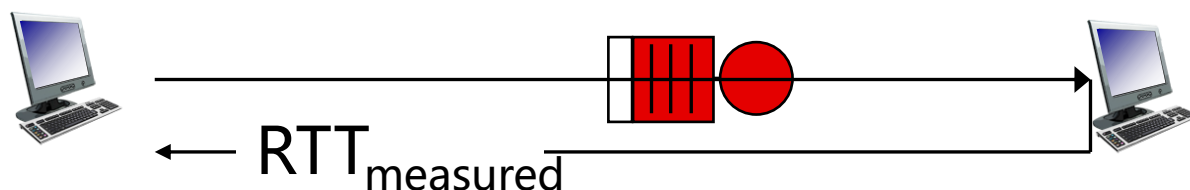
网络辅助拥塞控制:

- IP头部的TOS字段, 2个bit被网络路由器标记, 用于指示是否发生拥塞
- 拥塞指示被传送到接收主机
- 在接收方-到发送方的 ACK 中, 接收方(在 IP 数据报中看到了拥塞指示) 设置ECE bit, 指示发送方发生了拥塞
- 包括 IP (IP header ECN bit marking) 和 TCP (TCP header C,E bit marking)



基于延迟的拥塞控制

- 保持发送端到接收端管道 “仅足够满，但不全满”：保持瓶颈链路忙于传输，但避免高延迟/缓冲



$$\text{measured throughput} = \frac{\text{\# bytes sent in last RTT interval}}{RTT_{\text{measured}}}$$

基于延迟的方法:

- RTT_{min} – 观察到的 RTT 最小值 (非拥塞路径)
- 具有拥塞窗口 **cwnd** 的非拥塞吞吐量为 $\text{cwnd}/RTT_{\text{min}}$

如果测量的吞吐量 “非常接近” 非拥塞吞吐量

- 线性增加 **cwnd** (因为路径不拥塞) ;

否则, 如果测量的吞吐量 “远低于” 非拥塞吞吐量

- 线性减少 cwnd (因为路径拥塞)

第 3 章：提纲

3.1 概述和传输层服务

3.2 多路复用与解复用

3.3 无连接传输：UDP

3.4 可靠数据传输的原理

3.5 面向连接的传输：TCP

- 段结构
- 可靠数据传输
- 流量控制
- 连接管理

3.6 拥塞控制原理

3.7 TCP拥塞控制

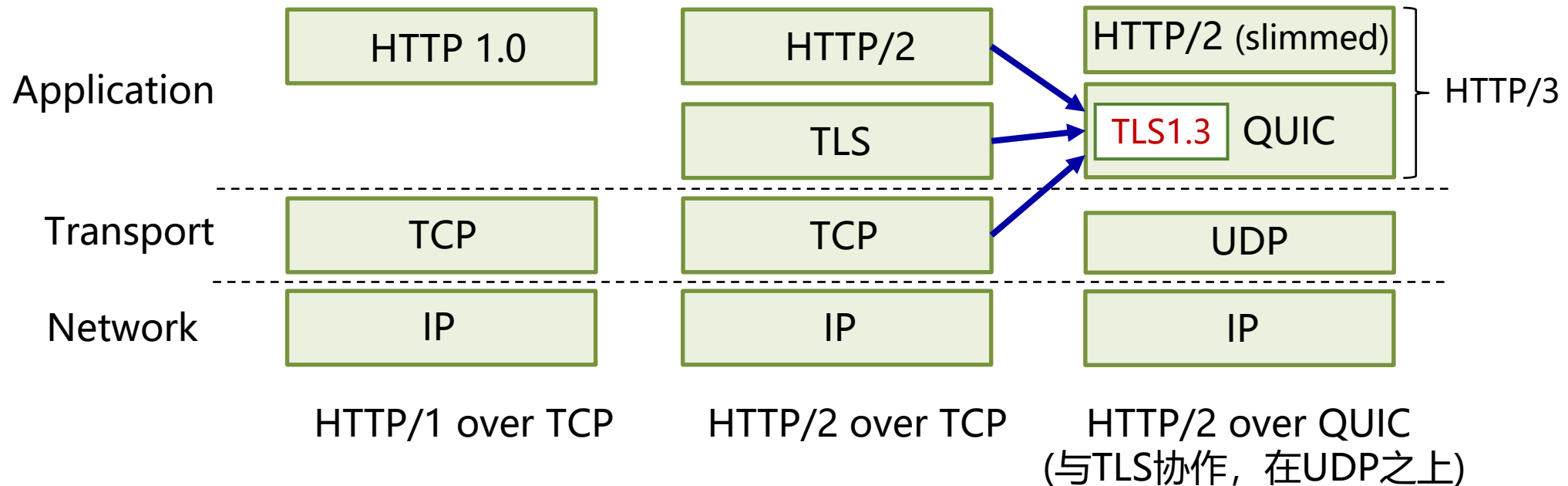
3.8 传输层功能的演化

传输层协议演化的必要性

- TCP, UDP: 支撑互联网发展40余年的2个传输层协议，所有的应用必须二选一
 - TCP：面向连接，可靠、保顺字节流，拥塞控制，流量控制机制
 - UDP：无连接、区分应用进程
- 出现越来越多的应用，TCP和UDP都无法很好地满足其需求
 - TCP：可靠性恢复实现代价大，效率低，拥塞控制机制假设丢失即拥塞，享受了TCP的服务品质，必须要忍受其实现服务方式的代价
 - UDP：仅提供应用进程的区分，应用需要自己实现可靠性恢复工作和拥塞控制工作，应用进程工作量太大
- 应用需求多样化、高速链路、DC网络和无线链路等新型网络大量采用
 - TCP和UDP都无法满足要求，传输层协议需要演化

QUIC: Quick UDP Internet Connections

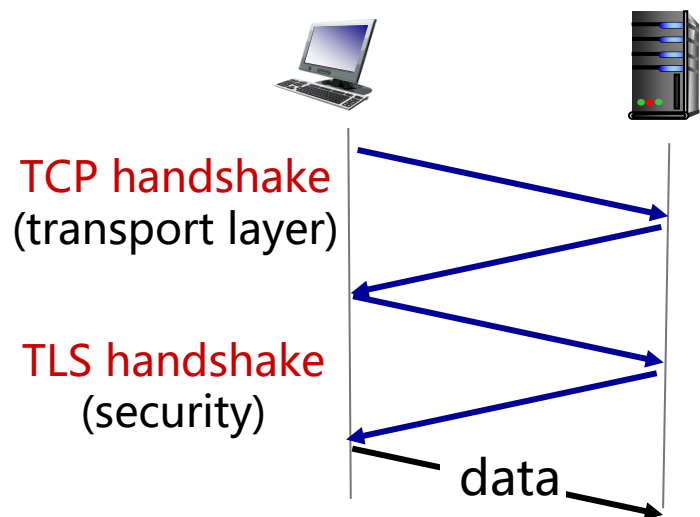
- 应用层协议, 在 UDP 之上
 - 提高HTTP性能
 - 部署在许多谷歌服务器、应用程序Chrome, 移动YouTube应用程序)



QUIC: Quick UDP Internet Connections

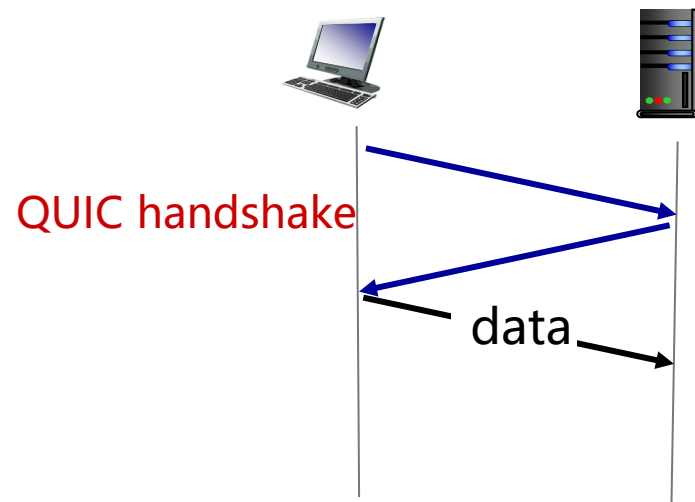
- TCP: 连接建立、差错控制、拥塞控制等, QUIC重新实现, 且是高效地实现
 - 差错控制和拥塞控制: “熟悉TCP的丢失检测和拥塞控制的读者会在这里找到于总所周知的TCP中的平行算法。” [from QUIC specification]
 - 连接建立: 可靠性 (初识序号), 拥塞控制 (窗口), 认证 (身份认证), 加密状态 (密钥) 在一个 RTT 内被建立
- 多个应用级别的 “流” 复用于一个 QUIC 连接之上, 提供复用功能
 - 每个流进行独立的可靠数据传输和实现安全性
 - 各个流服从公共的拥塞控制 (可插拔)

QUIC: 连接建立



TCP (可靠性、拥塞控制状态) +
TLS (身份验证, 加密状态)

- 2 次连续握手



QUIC: 可靠性, 拥塞控制,
身份验证, 加密状态

- 1 次握手

第三章总结 (1/2)

■ 传输层提供的服务

- 应用进程间的逻辑通信
 - V.s 网络层提供的是主机到主机的通信服务
- 互联网上传输层协议：UDP TCP

■ 多路复用和解复用

- 端口：传输层的SAP
- 无连接的多路复用和解复用
- 面向连接的多路复用和解复用

■ 实例1：无连接传输层协议 UDP

- 多路复用解复用
- UDP报文格式
- 检错机制：校验和

■ 可靠数据传输原理

- 问题描述
- 停止等待协议：
 - Rdt1.0 Rdt 2.0,2.1 ,2.2 Rdt 3.0

■ 流水线协议

- GBN, SR

回顾： rdt 1.0 – 3.0

- 假设：信道是可靠的
 - 1.0 – 可靠传输
- 假设：信道存在比特受损（差错）
 - 2.0 – ACK/NAK (去掉可靠假设，传输会出错)
 - 2.1 – 处理ACK/NAK错误，对发送分组标号（等0，等1）
 - 2.2 – NAK FREE (简化表述 $\text{NAK}_N = \text{ACK}_{(n-1)}$)
- 假设：比特差错+丢包
 - 3.0 – Timer (增加定时器，应对报文丢失情况)

第三章总结 (2/2)

■ 实例2：面向连接的传输层协议 - TCP

- 概述：TCP 特性
- 报文段格式
 - 序号，超时机制及时间
- TCP 可靠传输机制
- 重传，快速重传
- 流量控制
- 连接管理
 - 三次握手
 - 对称连接释放

■ 拥塞控制原理

- 端到端的拥塞控制
- 网络辅助的拥塞控制

■ TCP 的拥塞控制

- AIMD
- 慢启动
- 超时之后的保守策略

第三章重点（第六版）

- 复习题

- R3 R4 R7 R8 R9 R10 R11 R18

- 习题

- P15 P24 P25 P27 P40

第三章 展望

接下来的二章:

- 离开网络 “边缘” (应用层和传输层)
- 深入到网络的 “核心”
- 2个关于网络层的章
 - 数据平面
 - 控制平面